Study of Software Quality Improvement through Reliability Metrics Models and Root Cause Analysis Program

Muhammad Raees

Master's Student at Bahria University Islamabad

muhammadraees@mail.com

Abstract: Software quality is the most crucial requirement of any organization to survive in the current market scenario. The improvement of quality in software products depends on number of factors such as its architecture, quality metrics, reliability models and root cause analysis for defect handling. This paper focuses on the use of different reliability metrics to improve the reliability in software systems. Paper also emphasizes that removing defects for once is not beneficial for quality improvement, rather using a quality improvement program such as root cause analysis to get to base of defects and decrease the chances of occurrence of same defects again and again. Paper shows that effective root cause analysis program improves the learning process in any organization and helps in development of standards. This article provides an overview of software reliability metrics and root cause analysis program for the improvement of total software quality.

Keywords

Software Quality Improvement, Reliability Metrics, Root Cause Analysis.

1. Introduction

In the current market scenario everything is moving and mostly dependant on systems or software systems. The main objective of any system or software product is to satisfy its users. Software in an intangible thing, further it cannot be felt or touched. But it is needed to test the software systems to make effective use of software systems for its users. To test the software systems we need some evaluating or measuring criteria to ensure quality of system. Reliability links to software quality. That criteria is an essential parameter for the reliability of software systems. Software reliability has been a key for market survival of any company or its software product. So reliability of software system is one of the most crucial step in its success. It can be said that a software system that has low reliability is of no or little use to its end users. [1]

To increase the software quality, a root cause analysis program is executed. This main focus of this program to find the faults that are responsible for system failure and use the best practices to overcome these issues in the longer run to decrease the probability of faults occurring again and again. Here the figure-1 highlights quality improvement factors in any of software product. Software architecture and software quality metrics main blocks of software quality improvement. But models of software reliability and root cause analysis program directly influence the software quality. [2]

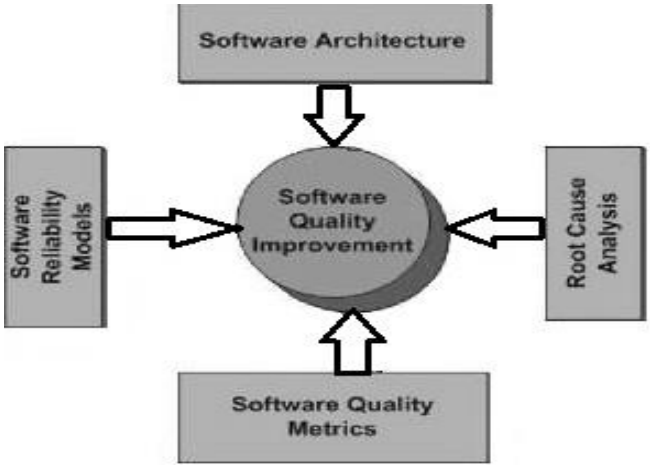

Figure-1. Software Quality improvement factors.

In this paper I will be discussing these two factors of quality improvement in software systems. Section 2 present some literature review about the software quality improvement. Section 3 explain the software reliability and importance of software reliability metrics. Section 4 list the reliability measurement techniques. Section 5 describes the role of effective root cause analysis program for software quality. Conclusion is present in the last section.

2. Literature Review

In [3] it is presented that software systems are always prone to errors. It is stated that reliability is categorized into three parts. First there is need to model the reliability measurements. It is noted that software reliability cannot be measured directly. This is done by applying some reliability metrics. Then we need to improve the modelling and measurements of reliability models by performing software development life cycle processes effectively. Identification of defects and their prevention directly results in software quality improvement. [4] Defect classification and its analysis is integral part of continuous quality improvement. It is almost impossible to deliver a software product that is hundred percent defect free and hundred percent reliable. In [5] it is presented that software is a complex entity and faults occur due to imperfect debugging. In [6] author states that defects are main reason for quality decline. So we need to extend our data collection to involve a root cause analysis process.

3. Software Reliability

The one IEEE definition about reliability says, “Under some defined set of conditions and a defined period of time, how much a software system is able to perform its designed functions providing desired output”. This definition states that software reliability is related to testing or measuring the output of software systems under certain conditions and specified time. This also stated that software system should also provide accurate results in that period of time. So software reliability can also be defined as “correct working of a software system under specified conditions in a specified time”. For a software system to work correctly in a specified period of time, it must be tested to find and then remove its bugs or errors. Software reliability is said to be composed of following activities. [7]

1. Fault prevention
2. Fault detection and removal

The main goal of software reliability to remove the bugs or errors in a software system and improve its usability for users. This paper focuses on software reliability measurement techniques.

Software reliability or reliability is defined as the probability of a system to perform is intended functions under defined conditions for a specified period of time. It is also defined as probability of system to work failure free in a given environment for a specified period of time. As the rate of failures or errors increases in any system the reliability of that system decreases. This makes an inversely proportional relationship between reliability and number of bugs in system. That can be defined using an equation

$$R(p) = \frac{1}{E}$$

- Where R (p) is probability of system being reliable.
- E is number of errors present in any software system.

Here software reliability can be said in the percentage of system being reliable. It is worth mentioning here to improve the reliability or quality of any system number of errors must be decreased. A software system is always prone to bug, therefore decreasing the number of bugs improves reliability which directly improves the quality of that system. The quality of system is continuous activity that is achieved by making highly reliable system. The one definition of quality also states that a software system at any given time has at least one bug. This definition highlights the importance of improving reliability and quality by continuous fault removal and continuously finding new faults that are present but not visible currently.

Software Reliability with respect to size and time

Software system errors belong to a big set. These may range to simple computational bug to a whole system failure issues. These range from a spelling mistake to in user interface to direct errors in the programming language. Software does not gets old, it does not gets rust, it does not deform. Software system will work as it is designed to work. Problems lies when over a period of time the software is upgraded and design changes are made intentionally or un-intentionally. Software errors are produced due to non-conformance to the requirement specifications. They may come due to ambiguous or misinterpreted requirements, poor design, incompetence code writing and inadequate testing. Below in figure-2 it is presented that error rate in each development process of software system with respect to time. It shows that during the requirement phase the error rate is very high, because the requirements are mostly unclear and ambiguous. With moving into design and code, the rate of defects decreases. But it increases again whenever there is an upgrade to the system. This is said to be the software systems will have high rate or defects and failures with respect to time because these are consistently upgraded. It is also worth mentioning here that software systems are prone to much more error that they are anticipated or idealized to have. [8]

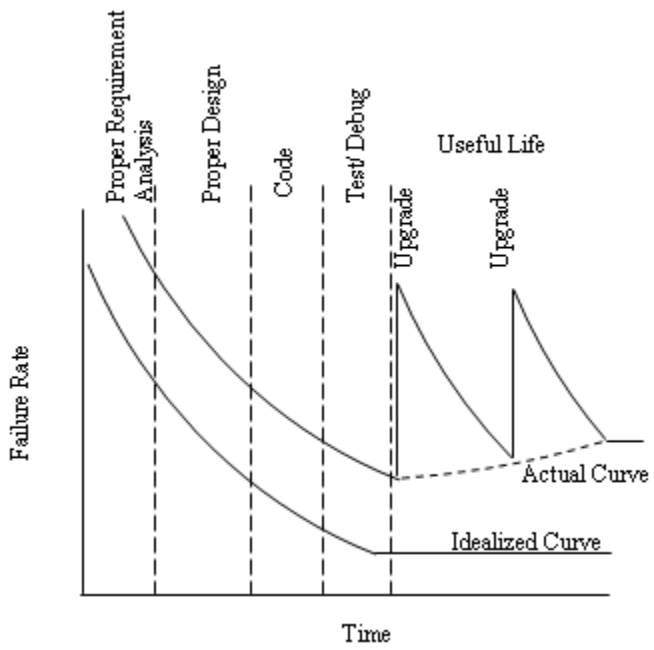

Figure-2. Failure rate with respect to time is software products.

While software reliability consistently decreases with respect to increase in complexity of software system. Which is shown below in figure-3.

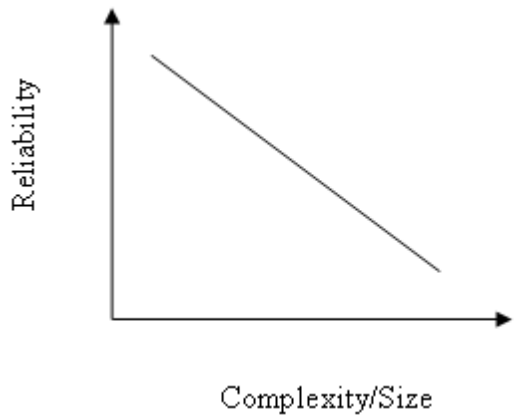

Figure-3. Reliability of software system decreases with increasing complexity.

Reliability Metrics

Metrics are measures. Reliability metrics are quantifiable measures of software reliability. Choosing a metric is dependant of type of system. Measuring reliability is difficult task. Some reliability metrics are defined which people have been using to measure the reliability of system. [9] Some of these discussed here:

Mean Time to Failure

It is defined as time between the continuous failures. Mean time with 500 means that failure occurs every 500 units of time. Time units are defined by the reliability measurer and it is dependent on the nature or system.

Mean Time between Failures

Mean time between failures is defined as the time between two successive failures plus the time to recover from this failure. It is repressed as:

$$mean\ time\ between\ failures = mean\ time\ to\ failure + mean\ time\ to\ repair$$

Rate of Failures

It is defined as number of failures during a specified period of time. It is also defined as the rate of occurrence of undesired events in specified period of time.

Mean time to Repair

After failures occur, it is needed to fix the software system. The time taken to fix the issues is called the mean time to repair which is also used as metric to measure software reliability.

Probability of Failure on Demand

It is the probability of software system that will fail when it is requested for a service. It is system failure when there is an input to system. It is an important measure for system that are safety critical and where demand of service is high.

Availability

It is probability of system that is available for used at given time. An availability of .98 means that system stays available 0.98 units of time in every 100 time units.

4. Software Reliability Measurement Techniques

We cannot measure the reliability of software system directly. We need to measure other characteristics of software system to define its reliability. Some reliability measurement metrics are discussed here.

Product Metrics

Product metrics are used to measure the conformance to requirement of any software system. These metrics shows how good the product is, by reporting on number of metric results. There are some metrics described here which are used for reliability measurements. [10]

Size

Size of software system reflects its complexity and development effort. Lines of Code or Lines of Code in thousands it an approach to measure software size. This indicates the effort and ease of maintenance of software system. Reliability of software system decreases when complexity of system increases.

Function Point

It is a metric to measure the functional points of software system. It comprise of count of inputs, outputs, class and interfaces between the components of software systems.

Test Coverage

This metric estimates the faults in software system, and ensures the reliability. How much the part of system has been tested and verified shows an indicator of reliability.

Complexity

As the size of software system increases, the complexity also increases. A software with small size may also be more complex that a larger software system having more dependencies and critical implementations. McCabe's complexity metrics are used to find the complexity of software systems.

Project Metrics

These describe the project characteristics. If there is good management of project is done, we are able to get highly reliable product. Project metrics are related to delivery of software products within budget and on time. The cost of product increases when software reliability is decreased due to in-efficient use of processes. Higher reliability is achieved by using best industry practices, risk management and management processes. This metric tell us about:

- No. of Software Developers
- Cost and Schedule
- Productivity
- Resource utilization

Process Metrics

Process metrics quantify the usefulness of software development process. They ensure that a product developed using best process models will have high reliability. It ensures the principle “build the product right”. Process metrics are used to evaluate, monitor and estimate the process of developing product. Process metrics define the best practiced processes produce highly reliable software products. They show

- Development time
- Development effort
- Effective defect removal in development
- Defects found in testing
- Maturity of process

Fault and Failure Metrics

A fault or defect is seen when developers make errors. There are metrics to measure failure free execution of software system. Problems are reported by users or testers are collected and analysed.

Requirement Reliability Metrics

Requirement are first step in software product development, and it specifies what a software system must contain. It tell the functionality that a system must have. To get highly reliable products it is worth to collect requirements effectively to minimize the chances of failures. There should be valid structure to save valuable information. Quality of requirements in detailed manner increases the reliability and eases the design phase. There should not be ambiguity or misinterpretation in requirements. Requirement reliability metrics evaluate the above factors.

Design and Code Reliability Metrics

Complexity size and modularization are factors affecting the design. As the design becomes complex, it is hard to understand and rate or errors is also increased. Reliability will also decrease if a software system is too complex with smaller or larger size. Design and code metrics are applicable to object oriented design.

Testing Reliability Metrics

Testing ensures that system has been built with conformance to requirements and it is performing the desired functionality accurately. This ensures that errors are decreased. Evaluating code and finding faults also improves reliability. Test cases ensure the effective verification of every requirement specified. [11]

5. Root Cause Analysis and Defect Elimination

Root-cause analysis is process applied to defect information to develop organizational understanding of the causes of a particular class of defects. It is done in a group of people reasoning about a specific defect.

Defect elimination is process to find out the defects in the software system and to remove those bugs to eliminate the chances of occurrence of defects again in the life time of software product. A "defect elimination program" is used by software development organizations is a structured process to eliminate the defects and make it reliable and consistent. It forms a part in larger picture of software quality improvement program along with software reliability models and other quality attributes. It is applied to all the wealthy efforts made within the organization to improve quality and reliability as whole.

The worth of effort is determined by comparing the effort to solve defects and its consequences. Consequences are defined using a risk matrix that categorizes severity and likelihood of occurrence of particular defect. Root cause analysis process is heart of defect elimination process. Root cause analysis process is a problem solving technique to identify and control the root causes of these defects. The effectiveness of this process depends on the quality of analysis and actions taken to resolve the issues. If the analysis done is very high level but the actions taken to resolve issues were poor the results may not be as good to eliminate the chances of defects appearing again and problems may appear again soon after. Again if the analysis is done with low quality and not addressing specific problems the result of best actions taken with respect to analysis results are not effective. Like above the result or outcome is not good solution and problem may appear again.

To use a process to give its best results we must ensure that everyone is using single approach and working towards the common goal to solve defects by analysing the defects to a high standard and applying actions to resolve defects corresponding to analysis. A common set of questions allow you to judge the process. These question may be like:

- Do you have a right process?
- Is it being used properly?
- Are actions are being taking properly?
- Are there resources available to solve problems?
- Does everyone knows what is expected from them?

By eliminating the defects the cost of running system also reduced. It improves the productivity and safety. That will directly improve the market profits of organization. It is not an un-common thing in any product to defects with respect to time. But the quality of that product depends on how these problems are detected and put into place to avoid their reoccurrence. That is whole goal of total quality improvement program. If we choose the same processes again and again for specific type of problems that occur again and again. We are not

learning anything and we are standing at the same point until we do not do anything different. In the current scenario of economic climate the organizations moving on with the defect elimination program can survive otherwise their competitors with efficient processes may leave them out.

In summary we can say that an efficient root cause analysis system is a fundamental to remove defect from software system to improve its quality. Here are some key elements of common root-cause analysis processes

- There should be a process for specific problem.
- Problem is clearly defined is measureable.
- Common understanding for problem root cause.
- Encouraging everyone to find unknown problems.
- Finding as many solutions and then picking most appropriate one.
- Solution filtering based on relevance to problem.
- Process should be easy to learn and adaptable to use easily.
- Proper closing of process to find best out of it.

A proverb about planning says “people don’t plan to fail, they simply fail to plan”. Sound planning is key to success. Effective planning and implementation of a defect elimination program is key part in software quality improvement. [12] Figure-4 shows a sample root cause analysis diagram that highlights the different entities involved in the defect cause and its handling.

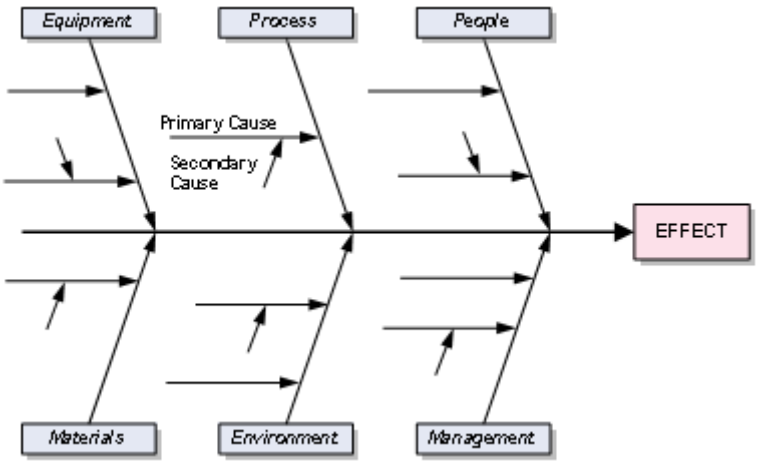

Figure-6. A general fish-bone diagram for root cause analysis program.

Conclusion

It is well known that software quality is huge area of research. Quality of software system is directly dependant to number of faults present in it. A mediocre designed system with no defects can be said to be the quality system. But to build a quality product it is mandatory to use a quality process. Thus by ensuring the quality of testing and reliability metrics to measure the quality of system, the system quality can be increased. Reliability metrics define current quality and predict for future. Reliability decreases if system gets faulty by defects. Removing these bugs in way that they do not occur again improves the quality of process. This is done by using a program for defect identification and removal. That program is called root cause analysis. By ensuring quality of process to identify bugs in any software systems and removing those bugs with the testing of system reliability using reliability metrics the overall

quality and robustness of system is increased. This paper also presents that software is a complex entity so reliability testing is also a hard goal to attain.

References

[1]. Musa, Iannino, “Software Reliability Engineering: Measurement, Prediction and Application”. Mc Graw Hill, 1987.

[2]. Michael R. Lyu, “Chapter 1”, “Software Reliability Engineering: A Roadmap”.

[3]. Mandeep Kaur. “A Review of various metrics used in software reliability”, “International Journal of Computer Science & Engineering Technology (IJCSET)”. ISSN: 2229-3345 Vol. 4. No. 07. Page 874-876. July 2013.

[4]. Arpita Mittal. “Defect Handling In Software Metrics “, “International Journal of Advanced Research in Computer and Communication Engineering”. ISSN: 2278–1021. Vol. 1. Issue 3. Page 167-170. May 2012.

[5]. Madhavi Mane. “Software reliability improvement with quality metric and defect tracking”, “International Journal of Application or Innovation in Engineering & Management (IJAIEM)”. Volume 3. Issue 1. ISSN 2319 – 4847. Page 364-369. January 2014.

[6]. Robert B. Grady. “Software Failure Analysis for High-Return Process Improvement Decisions”, “Hewlett-Packard Journal”. August 1996.

[7]. Goutam Kumar Saha. “Software Reliability Issues: Concept Map”, “IEEE Reliability Society 2009 Annual Technology Report”. 2009.

[8]. Aasia Quyoum, “Improving Software Reliability using Software Engineering Approach - A Review”, “International Journal of Computer Applications”. ISSN: 0975 – 8887. Vol. 10. No. 5. November 2010.

[9]. Vinay Tiwari1. “Open Source Software and Reliability Metrics”, “International Journal of Advanced Research in Computer and Communication Engineering”. Vol. 1. Issue 10. ISSN: 2278-1021. Page. 808-815. December 2012.

[10]. Vasilescu, B., Serebrenik. “By No Means: A Study on Aggregating Software Metrics”, “WETSoM’11”. ISBN: 978-1-4503-0593-8. Page 23-26. May 2011.

[11]. Gurpreet Kaur. “Software Reliability, Metrics, Reliability Improvement Using Agile Process”, “IJISET - International Journal of Innovative Science, Engineering & Technology”. Vol. 1. Issue 3. ISSN 2348 – 7968. Page 143-147. May 2014.

[12]. “Root Cause Analysis and Defect Elimination”. Available: http://www.thereliabilityblog.com/2013/12/03/root-cause-analysis-and-defect-elimination/. May 2015.